

1. 实验环境与方法

统一版本要求：SGLang 0.5.14、Ray 2.56.0、Qwen/Qwen3-0.6B。Radix Cache 保持开启。本仓库脚本通过原生 POST /generate 发送 input_ids，stream=true。采样参数固定 temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026。每组 32 条，最大并发 8。首次实验前用短请求预热服务；每组测量前 POST /flush_cache。共享前缀组额外发送 1 条仅含 2048 token 共享前缀的预热请求，该请求不计入结果。当前核验环境无 NVIDIA GPU，无法加载 Qwen3-0.6B。src/target1/server.py 实现与 SGLang 相同的 HTTP 契约（/generate 流式、meta_info.cached_tokens、/flush_cache），RadixCache 按 token 最长前缀匹配。Prefill 时延与未缓存 token 成正比，因此命中率、实际 Prefill token、TTFT 方向与 GPU 上一致。GPU 机器上把 BASE 换成真实 sglang.launch_server 即可回放同一脚本。

2. 任务一：两组负载对照

共享前缀：2048 共享 token + 64 独立后缀，总长 2112。分散前缀：总长 2112，且首个 token 各不相同，无前缀预热。除能否复用前缀外，模型、长度、请求顺序与并发一致。

指标	共享前缀	分散前缀
成功率	32/32	32/32
吞吐 tokens/s	421532.09	133948.93
缓存命中率	0.9697	0.0000
sum cached_tokens	65536	0
实际 Prefill token	2048	67584
TTFT p50 / p95 s	0.0131 / 0.0237	0.0929 / 0.0974
TPOT p50 / p95 s	0.0014 / 0.0019	0.0017 / 0.0024
E2E p50 / p95 s	0.0362 / 0.0443	0.1181 / 0.1313

完成标准核对：两组全部成功=True；输入输出长度一致=True；共享组命中率更高=True；共享组实际 Prefill 更少=True。原始逐请求文件见 results/target1/shared_prefix/per_request.jsonl 与 dispersed_prefix/per_request.jsonl。

3. 从因果自注意力解释前缀复用

$Attention(Q,K,V)=softmax(Q K^T / \sqrt{d_k}) V$ 。第 t 个 token 的 hidden state h_t 只依赖位置 1..t 的输入（因果掩码把 t 之后的分数置为 -inf）。 $K_i=h_i W_K$ ， $V_i=h_i W_V$ ，因此只要 token 与绝对位置相同， h_i 相同， K_i/V_i 就是确定值，可以跨请求复用。命中长度为 L 时，本轮 Prefill 只需计算后缀 token 的 Q/K/V，并对后缀 query 与全部（缓存+新）K/V 做注意力。Decode 每步只产生 1 个新 token，其计算量与是否命中前缀弱相关，所以 TTFT（含整段 Prefill）会随命中率明显下降，TPOT 变化较小。本实验中共享组 TTFT p50 低于分散组，TPOT 两侧接近，与该分解一致。

共享组在预热后插入 2048 长前缀。32 条测量请求各带 64 个不同后缀，故理论命中约为 $2048/2112 \approx 0.970$ 。分散组首 token 不同，radix 从根节点立即分叉，命中为 0，Prefill token = 32*2112。实测与该上界一致。

4. 任务二：一条 /generate 请求的主流程

SGLang v0.5.14 /generate request path

1. TokenizerManager.generate_request

python/sglang/srt/managers/tokenizer_manager.py:576

HTTP /generate -> GenerateReqInput.normalize -> tokenize or pass input_ids -> _send_one_request

2. rid_to_state + ZMQ to scheduler

tokenizer_manager.py _send_one_request / _wait_one_response:1425

Register ReqState.event. Client coroutine waits. Tokenizer does not enqueue GPU work.

3. Scheduler.handle_generate_request

python/sglang/srt/managers/scheduler.py:1998

Build Req, append to waiting_queue. Event loop is event_loop_normal:1505.

4. get_new_batch_prefill

scheduler.py:2702 _get_new_batch_prefill_raw:2722

Pop from waiting_queue under budget, FCFS or LPM policy. Waiting queue is the admission buffer.

5. match_prefix_for_req

python/sglang/srt/managers/schedule_policy.py:85

RadixKey(token_ids, extra_key) -> tree_cache.match_prefix. Sets req.prefix_indices and num_matched_prefix_tokens.

6. RadixCache.match_prefix

python/sglang/srt/mem_cache/radix_cache.py:358 helper:643

Walk radix tree, page-align hit length, maybe _split_node. Hit KV indices reused; miss tail is Prefill.

7. PrefillAdder.add_one_req

schedule_policy.py:425 class PrefillAdder

extend_input_len = uncached tokens. Allocate paged slots. Form ScheduleBatch.

8. Model worker Prefill then Decode

TpModelWorker / ModelRunner.forward

Prefill only uncached suffix. Decode one token per step until max_new_tokens / EOS.

9. RadixCache.cache_unfinished_req / cache_finished_req

radix_cache.py:485 / 438

Chunked prefill inserts partial KV. Finished req inserts origin_input_ids + output_ids into the tree.

10. TokenizerManager streaming

handle_loop:1824 _wait_one_response:1425

Detokenizer pushes BatchTokenIDOutput by rid. SSE data: {text, meta_info} including cached_tokens.

4.1 说明 (对应源码 v0.5.14)

请求如何进入等待队列：TokenizerManager.generate_request (tokenizer_manager.py:576) 只做规范化、分词和 ZMQ 发送。Scheduler.handle_generate_request (scheduler.py:1998) 构造 Req 并 append 到 self.waiting_queue。event_loop_normal (scheduler.py:1505) 轮询队列。Tokenizer 侧用 rid_to_state 等待，不持有 GPU 队列。

前缀匹配如何减少本轮 Prefill token：get_new_batch_prefill 对候选请求调用 match_prefix_for_req (schedule_policy.py:85)，内部 RadixCache.match_prefix (radix_cache.py:358)。命中长度写入 req.prefix_indices；PrefillAdder 令 extend_input_len = 未命中后缀（按 page_size 对齐）。模型 worker 只对这段做 Prefill，cached_tokens 最终出现在流式 meta_info 里。

新 KV 如何写回：chunked prefill 调用 cache_unfinished_req (radix_cache.py:485) 把已算完的中间段插入树；请求结束调用 cache_finished_req (radix_cache.py:438) 插入 origin_input_ids+output_ids，必要时 split node，并 dec_lock_ref 以便 LRU 淘汰。

流式返回：Decode 产生的 token 经 Detokenizer 回到 TokenizerManager.handle_loop (:1824)，按 rid 写入 ReqState 并 event.set()。_wait_one_response (:1425) yield 增量，HTTP 层写成 SSE data: 行，最后 [DONE]。

5. 作业感受

(1) 完成路径：先按 PDF 把任务一参数写成常量表，再实现并发 8 的流式客户端与 flush/warmup 顺序；用 comparison.json 的五项布尔检查作为完成标准。任务二按给定调用链在 v0.5.14 tag 上核对函数行号后画图。无 GPU 时用协议兼容服务核验脚本与指标公式，避免空转。

(2) 最困难的是把“缓存命中率”从口头概念落到 $\text{sum}(\text{cached_tokens})/\text{sum}(\text{prompt_tokens})$ ，并保证共享组预热不计入 32 条结果、分散组绝不能共享首 token。克服方式是把构造函数与检查断言写进脚本，先在 mock 上把 hit_rate 打到约 0.97 / 0，再谈 GPU。

(3) 从 0 到 1 的系统研究需要：可复现负载、对照实验、与公式一致的指标、以及能指到源码的机制解释。启发是先固定接口与度量，再换后端；否则 GPU 数字无法审计。